

Содержание

1	Собственная реализация хеш-таблицы	2
1.0.1	Общая архитектура	2
1.0.2	Разрешение коллизий	2
1.0.3	Удаление элементов	2
1.0.4	Resize и rebuild	3
1.0.5	Размер таблицы	3
1.0.6	Поддерживаемые операции	3
1.1	Тестирование реализации	4
2	Сравниваемые реализации	5
3	Benchmark results	6
3.1	Insert sequential	6
3.2	Insert random	8
3.3	Get existing sequential	10
3.4	Get existing random order	12
3.5	Get missing keys	14
3.6	Delete	16
3.7	Get after deletions	18
3.8	Insert after deletions	20
3.9	Mixed workload	22
3.10	Общий вывод по benchmark-ам Hash Table	24

Собственная реализация хеш-таблицы

В рамках работы была реализована собственная структура данных типа hash table как на Python, так и на C++. Обе реализации построены на одной и той же концепции и отличаются только языковыми особенностями.

В рамках курсовой работы я реализовал собственную структуру данных типа hash table с использованием open addressing. Основные характеристики реализации:

Общая архитектура

Структура данных использует **три параллельных массива**:

- массив ключей (`keys`)
- массив значений (`values`)
- массив состояний ячеек (`states`)

Каждая ячейка может находиться в одном из трёх состояний:

- `EMPTY` — ячейка никогда не использовалась
- `OCCUPIED` — ячейка содержит элемент
- `DELETED` — элемент был удалён (tombstone)

Такое разделение позволяет эффективно реализовать probing и корректно обрабатывать удаление элементов без нарушения цепочек поиска.

Разрешение коллизий

Для разрешения коллизий используется **open addressing с probing**:

- при вставке и поиске вычисляется начальный индекс по хэшу ключа
- далее выполняется последовательный обход (probing) по таблице
- используется схема **double hashing**

Таким образом, все операции (`insert` , `get` , `erase`) сводятся к обходу массива по определённой последовательности индексов.

Удаление элементов

Операция удаления (`erase` / `delete`) реализована через установку состояния `DELETED` , а не через физическое удаление элемента.

Однако накопление таких ячеек со временем ухудшает производительность, поэтому требуется дополнительная логика очистки.

Resize и rebuild

В реализации предусмотрено несколько сценариев, при которых происходит **перестроение таблицы**:

1. Рост таблицы (resize)

При достижении заданного **load factor** таблица увеличивается в размере, чтобы сохранить амортизированную сложность операций.

2. Очистка tombstones

Если количество **DELETED** ячеек становится слишком большим, выполняется rebuild, при котором:

- создаётся новая таблица
- все элементы перехешируются
- **DELETED** полностью исчезают

3. Комбинированный случай

В некоторых ситуациях rebuild может происходить одновременно из-за роста и из-за накопления удалённых элементов.

Таким образом, в реализации есть несколько условий, при которых таблица перестраивается, что напрямую влияет на производительность.

Размер таблицы

Размер таблицы поддерживается как степень двойки.

Это даёт несколько преимуществ:

- упрощение вычисления шага probing
- более эффективная работа на уровне процессора

Поддерживаемые операции

Реализация поддерживает стандартный набор операций:

- **insert** / **put** — вставка элемента
- **get** — поиск по ключу
- **erase** / **delete** — удаление элемента

Все операции имеют амортизированную сложность $O(1)$, но на практике производительность зависит от:

- load factor
- количества **DELETED** ячеек
- распределения хэшей
- выбранной стратегии probing

Тестирование реализации

Для проверки корректности реализации hash table был разработан набор тестов как для Python, так и для C++ версии.

Тесты покрывают основные аспекты работы структуры:

- базовые операции: вставка (`put`), поиск (`get`), удаление (`erase/delete`), проверка наличия ключа, корректный подсчёт размера;
- обработку коллизий (в том числе искусственно созданных);
- корректность работы с `DELETED` ячейками (tombstones), включая сохранение цепочек поиска и повторное использование удалённых слотов;
- поведение при изменении размера таблицы (`resize`) и при перестроении (`rebuild`);
- корректность работы после последовательностей вставок и удалений.

Также проверяются дополнительные свойства, такие как:

- обновление существующих ключей,
- работа с отсутствующими ключами,
- корректность копирования структуры.

Таким образом, тестирование покрывает как базовую функциональность, так и ключевые особенности реализации open addressing hash table.

Сравниваемые реализации

В качестве эталонных реализаций для сравнения были выбраны встроенный словарь Python `dict` и библиотечная структура `boost::unordered_flat_map` для C++.

Выбор `dict` обусловлен тем, что это стандартная и наиболее часто используемая реализация хеш-таблицы в Python. В отличие от пользовательской реализации, написанной непосредственно на Python, `dict` реализован внутри CPython на языке C. Поэтому основные операции — вставка, поиск и удаление — выполняются не как последовательность инструкций Python-байткода, а внутри низкоуровневой оптимизированной реализации. Это существенно снижает накладные расходы, связанные с интерпретацией циклов, вызовами методов, проверками типов и обращениями к пользовательским объектам.

Кроме того, `dict` является одной из ключевых структур данных языка Python, поэтому его реализация оптимизирована под типичные сценарии использования. В частности, словарь эффективно работает с часто используемыми типами ключей, такими как целые числа и строки. По этой причине `dict` является сильным baseline: сравнение с ним показывает не только алгоритмическую корректность собственной реализации, но и разницу между учебной реализацией на Python и промышленной встроенной структурой данных.

Для C++ в качестве библиотечного аналога был выбран `boost::unordered_flat_map`. Стандартная библиотека C++ содержит контейнер `std::unordered_map`, однако он не является наиболее близким аналогом для данной работы. `std::unordered_map` обычно использует bucket-based/node-based организацию, при которой элементы хранятся в отдельных узлах и распределяются по bucket-ам. Такая архитектура отличается от собственной реализации, построенной на open addressing, и хуже подходит для сравнения именно с точки зрения организации памяти и поведения при probing.

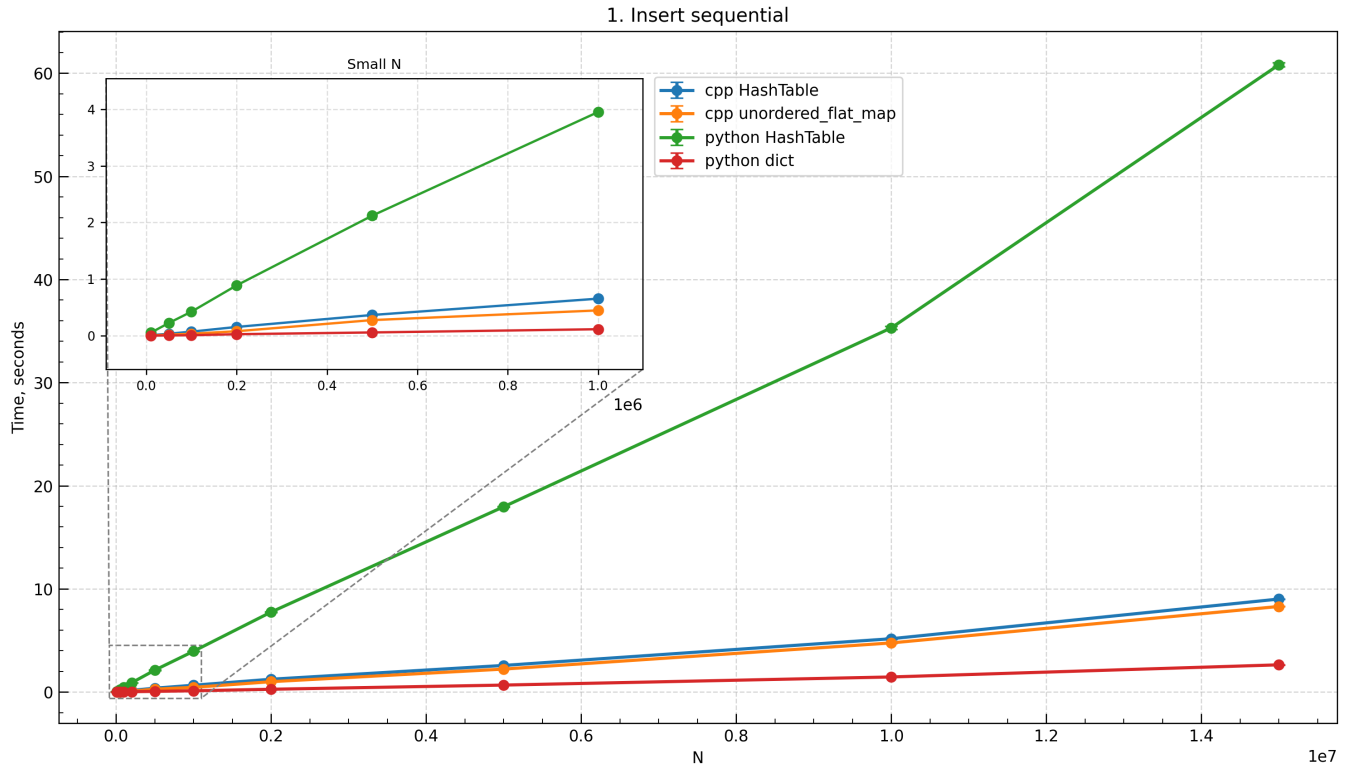
`boost::unordered_flat_map`, напротив, относится к классу flat hash map и использует более компактную организацию хранения элементов. По архитектурной идее он ближе к собственной реализации: данные размещаются более плотно, а разрешение коллизий связано с обходом ячеек внутри таблицы, а не с переходами по отдельным узлам. Благодаря этому сравнение с `boost::unordered_flat_map` является более содержательным: различия в производительности можно связывать с конкретными деталями реализации, такими как layout памяти, стратегия probing, обработка удалений, политика расширения таблицы и кэш-локальность.

Таким образом, выбранные реализации представляют собой индустриально оптимизированные аналоги хеш-таблиц для соответствующих языков. Python `dict` показывает уровень производительности встроенной низкоуровневой реализации в CPython, а `boost::unordered_flat_map` — уровень производительности специализированной C++ flat hash map. Это позволяет сравнивать собственные реализации не только с точки зрения асимптотической сложности, но и с точки зрения практических факторов: накладных расходов языка, организации памяти, эффективности работы с кэшем и качества низкоуровневой оптимизации.

Benchmark results

Insert sequential

В данном benchmark-case рассматривается последовательная вставка элементов в хеш-таблицу. Для каждого значения N в структуру данных вставляются ключи `0, 1, 2, ..., N - 1`, при этом значение совпадает с ключом. Для собственной реализации вызывается операция `put(key, key)`, для встроенной или библиотечной реализации используется стандартная операция вставки. Таким образом, данный тест измеряет стоимость массового заполнения изначально пустой хеш-таблицы.



note

1. Insert sequential

	N	cpp_HashTable	unordered_flat_map	python_HashTable	dict
5	1000000	~0.631 s	~0.447 s	~3.956 s	~0.103 s
6	2000000	~1.239 s	~1.011 s	~7.761 s	~0.247 s
7	5000000	~2.573 s	~2.215 s	~17.993 s	~0.652 s
8	10000000	~5.098 s	~4.792 s	~35.415 s	~1.454 s
9	15000000	~9.002 s	~8.289 s	~60.793 s	~2.633 s

По графику и таблице видно, что все четыре реализации демонстрируют близкое к линейному поведение при увеличении N . Это соответствует теоретической оценке для хеш-таблицы: одна вставка в среднем выполняется за амортизированное $O(1)$, поэтому серия из N вставок имеет ожидаемую сложность $O(N)$. Пери-

одические расширения таблицы не меняют общий порядок роста, поскольку их стоимость распределяется по большому числу обычных вставок.

Особенность данного benchmark-case состоит в том, что последовательная вставка целочисленных ключей является благоприятным сценарием для собственной реализации. Индекс ячейки вычисляется как $\text{hash}(\text{key}) \% \text{capacity}$, а размер таблицы поддерживается степенью двойки. Если считать, что для целых чисел $\text{hash}(\text{key}) = \text{key}$, то ключи 0, 1, 2, ... последовательно попадают в разные позиции таблицы. Кроме того, таблица расширяется до сильного заполнения, поэтому длинные последовательности probing практически не возникают.

Следовательно, в данном случае почти не проверяется эффективность разрешения коллизий. Benchmark в первую очередь показывает базовую стоимость операции вставки, стоимость периодического `resize` и влияние константных факторов. Поэтому форма графиков у всех реализаций похожа, но линии находятся на разных уровнях.

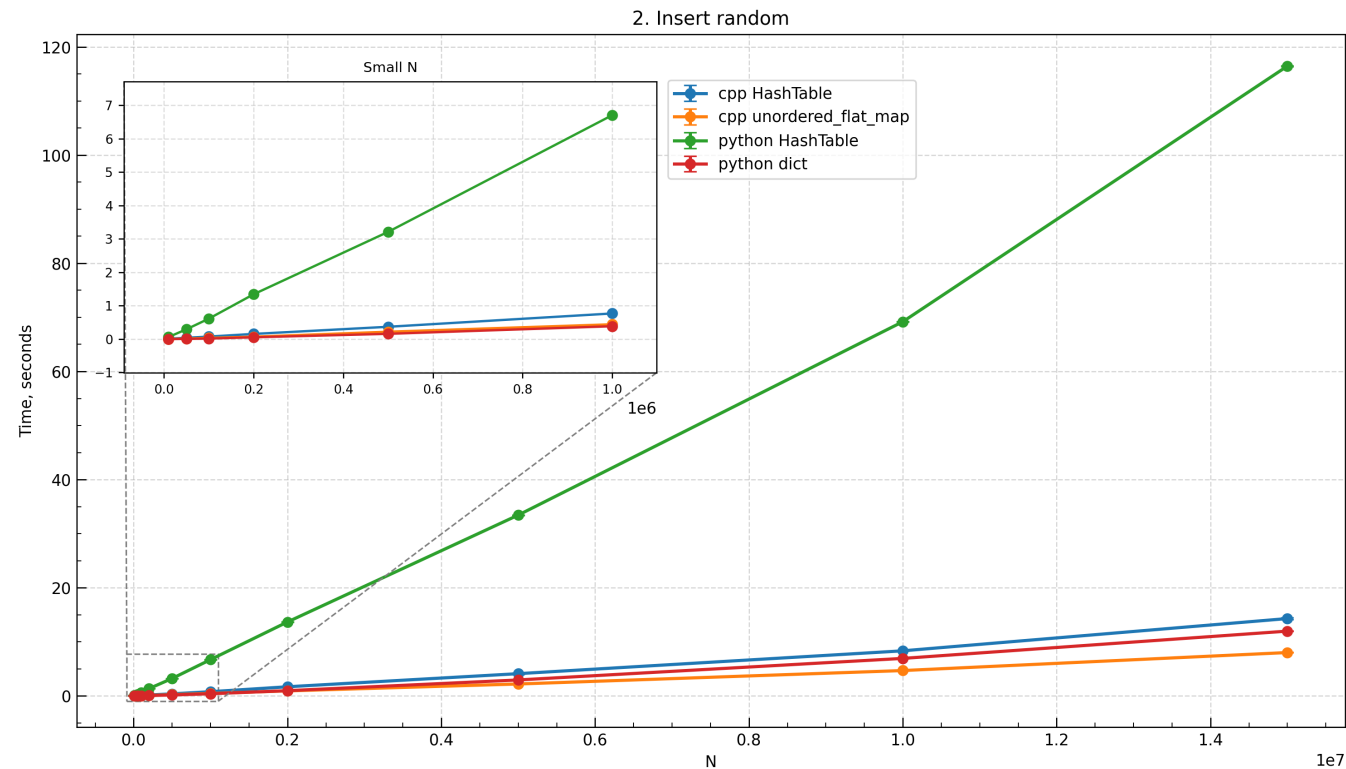
Наиболее заметный разрыв наблюдается между собственной реализацией на Python и встроенным `dict`. Это объясняется тем, что собственная реализация выполняет вставку, поиск свободной ячейки, проверки состояний и обращения к внутренним массивам на уровне Python-кода. Встроенный `dict`, напротив, использует низкоуровневую реализацию внутри CPython. Поэтому при одинаковом среднем порядке сложности стоимость одной вставки у собственной Python-реализации значительно выше.

C++-реализации находятся ближе друг к другу, поскольку обе выполняются как скомпилированный код. При этом библиотечная реализация показывает преимущество за счет более тщательной оптимизации, лучшей организации памяти и меньших постоянных расходов. Разрыв между ними заметен, но существенно меньше, чем между собственной Python-реализацией и `dict`.

Таким образом, benchmark-case `Insert sequential` подтверждает ожидаемое амортизированное линейное поведение хеш-таблиц при массовой вставке. Для собственной реализации этот сценарий является благоприятным, поскольку практически не содержит коллизий. Поэтому различия между реализациями здесь связаны в основном не с асимптотикой, а с постоянными факторами: уровнем реализации, накладными расходами выполнения, организацией памяти и степенью библиотечной оптимизации.

Insert random

В данном benchmark-case измеряется время вставки случайных уникальных ключей в изначально пустую хеш-таблицу. Для каждого значения N заранее генерируется набор из N различных случайных ключей, после чего в измеряемой части benchmark-а выполняется только вставка этих ключей в структуру данных. Значение, как и в предыдущем случае, совпадает с ключом. Генерация случайных чисел не входит в измеряемое время, поэтому результат отражает именно стоимость операций вставки.



note

1 2. Insert random

	N	cpp_HashTable	unordered_flat_map	python_HashTable	dict
5	1000000	~0.767 s	~0.442 s	~6.703 s	~0.387 s
6	2000000	~1.634 s	~0.912 s	~13.675 s	~0.948 s
7	5000000	~3.969 s	~2.176 s	~33.437 s	~2.930 s
8	10000000	~8.309 s	~4.655 s	~69.186 s	~6.921 s
9	15000000	~14.075 s	~7.974 s	~116.395 s	~11.943 s

По графику видно, что общий рост времени у всех реализаций остается близким к линейному. Это соответствует ожидаемой амортизированной сложности хеш-таблицы: одна вставка в среднем выполняется за $O(1)$, а серия из N вставок — за $O(N)$. Однако по сравнению с кейсом `Insert sequential` время выполнения заметно увеличивается, особенно у собственной Python-реализации.

Главное отличие от последовательной вставки состоит в распределении ключей. В предыдущем benchmark-

case ключи `0, 1, 2, ..., N - 1` при вычислении индекса `hash(key) % capacity` фактически попадали в разные соседние ячейки, поэтому коллизии почти не возникали. В данном случае ключи случайные, поэтому их начальные позиции в таблице также распределяются случайно. Из-за этого часть вставок попадает в уже занятые ячейки, и структуре приходится выполнять probing.

При этом деградации асимптотики не происходит. Таблица расширяется при достижении заданного load factor, поэтому заполненность не становится слишком высокой. Следовательно, средняя длина probing-последовательности остается ограниченной, и операция вставки по-прежнему имеет амортизированную сложность $O(1)$. Но константная стоимость одной вставки становится больше: вместо прямого попадания в свободную ячейку иногда приходится проверять несколько позиций.

Случайный порядок ключей также хуже с точки зрения кэш-локальности. При последовательной вставке обращения к ячейкам таблицы более предсказуемы и часто расположены ближе друг к другу. При случайных ключах обращения распределены по таблице менее регулярно, поэтому процессор хуже использует загруженные cache line. Это не меняет асимптотику, но увеличивает реальные постоянные расходы.

Наиболее сильно это проявляется у собственной Python-реализации. Для каждой вставки она выполняет поиск слота, проверки состояний, вычисление шага probing и обращения к внутренним массивам на уровне Python-кода. Когда к этим накладным расходам добавляются реальные коллизии и менее локальные обращения к памяти, время выполнения заметно растет. Поэтому линия `python HashTable` остается самой высокой и растет значительно быстрее остальных по абсолютным значениям.

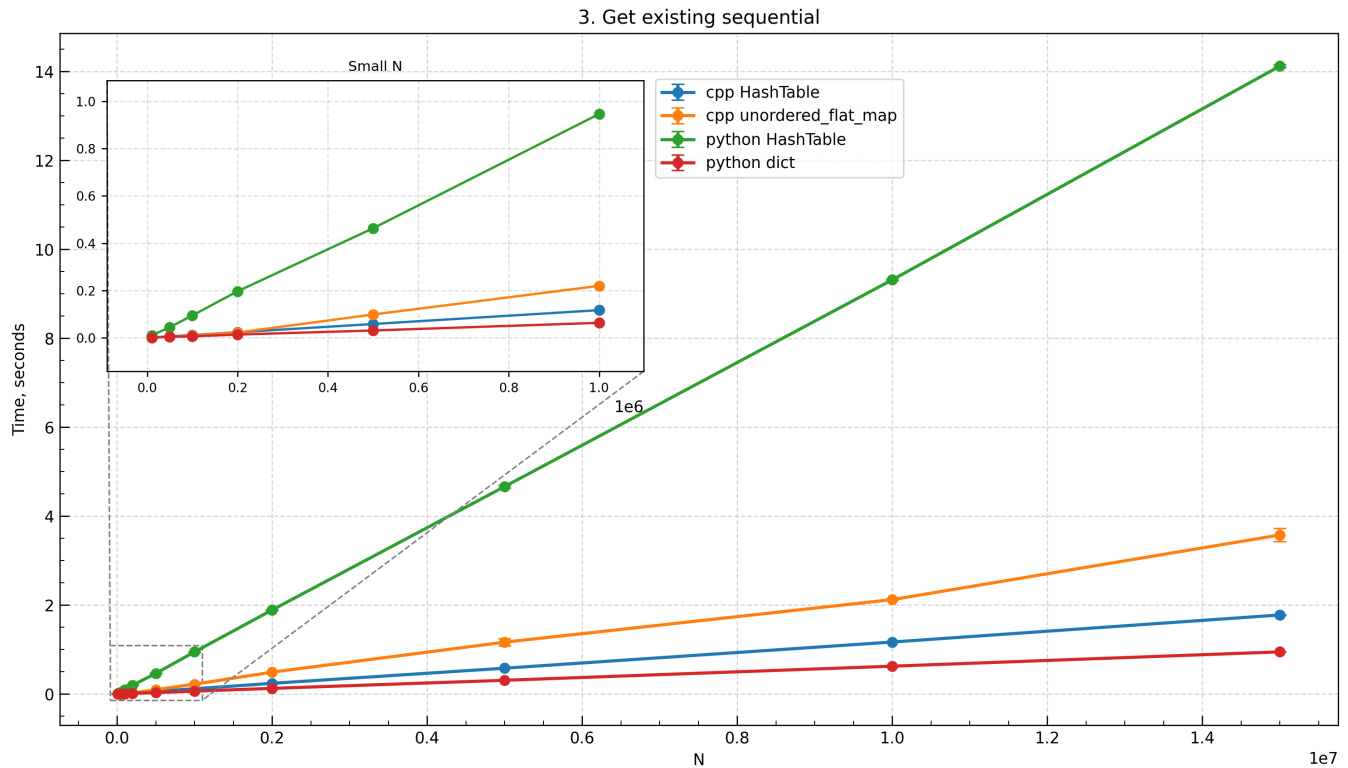
Встроенный `dict` также работает медленнее, чем в случае последовательной вставки, но разрыв с собственной Python-реализацией остается очень большим. Это объясняется тем, что `dict` выполняет основную работу внутри оптимизированной реализации CPython. Даже при случайном распределении ключей он эффективнее обрабатывает коллизии, лучше использует внутреннюю организацию памяти и имеет меньшие накладные расходы на одну операцию.

Среди C++-реализаций `unordered_flat_map` показывает лучший результат. В этом benchmark-case его преимущество над собственной C++-реализацией заметнее, чем при последовательной вставке. Это связано с тем, что случайные ключи активнее проверяют качество probing, layout памяти и обработку коллизий. Библиотечная flat hash map лучше оптимизирована для таких сценариев, поэтому имеет меньшие постоянные расходы.

Таким образом, benchmark-case `Insert random` показывает более реалистичный сценарий вставки в хеш-таблицу, чем последовательная вставка. Все реализации сохраняют ожидаемый линейный рост для серии из `N` вставок, однако случайное распределение ключей увеличивает количество коллизий, ухудшает кэш-локальность и повышает константную стоимость операций. Поэтому различия между реализациями становятся более заметными, особенно между собственной Python-реализацией и оптимизированными встроенными или библиотечными структурами.

Get existing sequential

В данном benchmark-case измеряется время поиска уже существующих элементов в хеш-таблице. Для каждого значения N структура данных заранее заполняется ключами $0, 1, 2, \dots, N - 1$, после чего в измеряемой части benchmark-а выполняется последовательный поиск этих же ключей. То есть измеряется не построение таблицы, а только выполнение операций поиска по уже существующим ключам.



note

1 3. Get existing sequential

2

3	N	cpp_HashTable	unordered_flat_map	python_HashTable	dict
4					
5	1000000	~0.117 s	~0.216 s	~0.941 s	~0.063 s
6	2000000	~0.238 s	~0.490 s	~1.898 s	~0.126 s
7	5000000	~0.579 s	~1.292 s	~4.668 s	~0.308 s
8	10000000	~1.164 s	~2.114 s	~9.310 s	~0.623 s
9	15000000	~1.774 s	~3.402 s	~14.122 s	~0.944 s

По графику видно, что все реализации снова демонстрируют близкое к линейному поведение при увеличении N . Это ожидаемый результат: поиск одного существующего ключа в хеш-таблице в среднем выполняется за $O(1)$, поэтому последовательность из N поисков имеет ожидаемую сложность $O(N)$.

Данный benchmark-case является благоприятным для собственной реализации. Таблица предварительно заполняется последовательными целочисленными ключами, и затем поиск выполняется в том же последовательном порядке. При предположении, что $\text{hash}(\text{key}) = \text{key}$, ключи хорошо распределяются по таблице, а

поиск большинства элементов завершается уже на первой проверяемой позиции. Поэтому длинные probing-последовательности практически не возникают.

По сравнению с операциями вставки этот кейс проще: при поиске уже существующих элементов не требуется добавлять новые пары, изменять размер таблицы или выполнять `resize`. Структура уже построена, и benchmark измеряет только стоимость обращения к ключу и проверки соответствующей ячейки. Поэтому абсолютные времена заметно меньше, чем в benchmark-case на вставку.

Наиболее медленной снова оказывается собственная реализация на Python. Даже при отсутствии существенных коллизий каждый поиск выполняется на уровне Python-кода: вызывается метод `get`, внутри него вызывается поиск слота, выполняются проверки состояния ячейки, сравнение ключей и обращения к внутренним массивам. Эти действия имеют заметные накладные расходы, которые при миллионах повторений дают большой разрыв по времени.

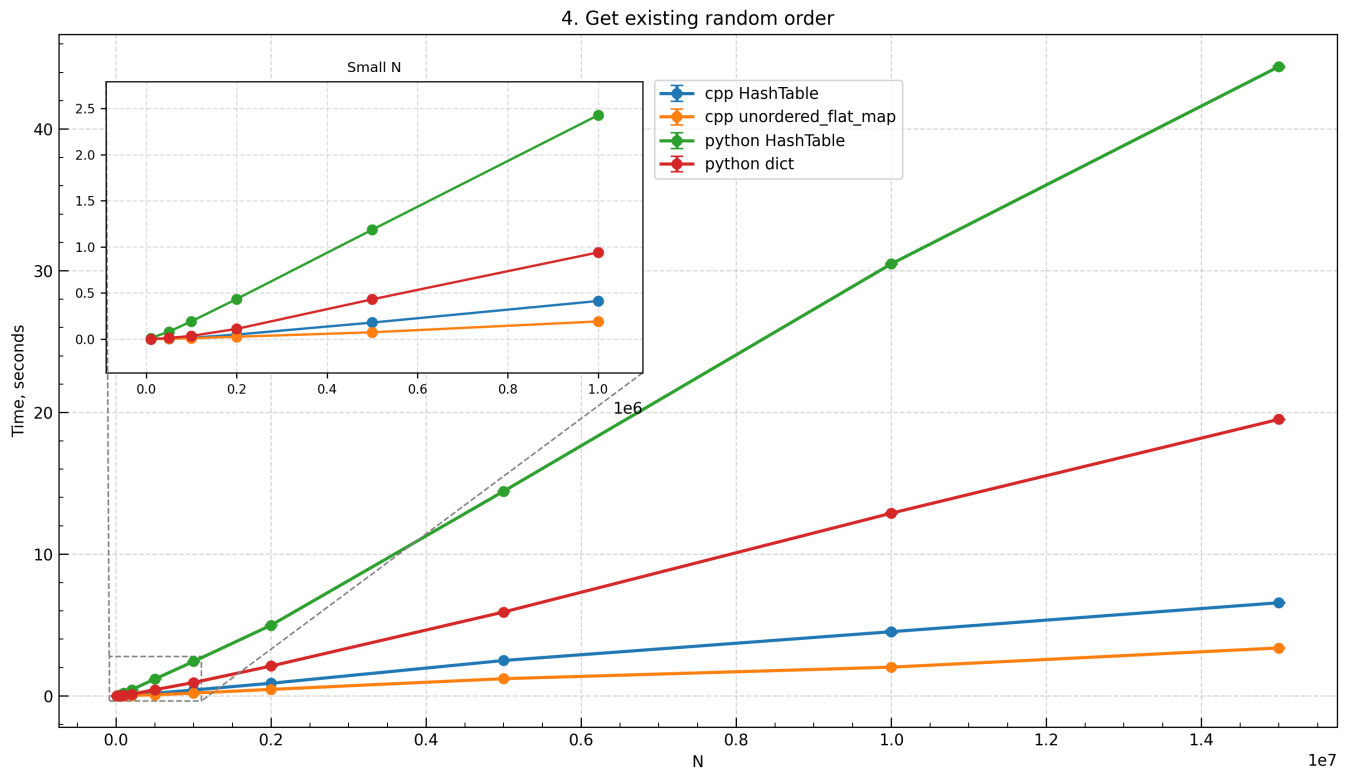
Встроенный `dict` работает значительно быстрее, так как операция поиска выполняется внутри низкоуровневой реализации CPython. При этом важно, что в данном кейсе `dict` также находится в благоприятном режиме: все ключи существуют, а их поиск не требует обработки отсутствующих значений или длинных последовательностей пробирования.

Интересный результат наблюдается среди C++-реализаций: собственная C++ HashTable оказывается быстрее `unordered_flat_map`. Это можно объяснить тем, что данный кейс очень хорошо подходит именно для простой собственной реализации. Последовательные ключи при текущей схеме хеширования почти напрямую попадают в свои позиции, поэтому поиск сводится к небольшому числу простых действий: вычислить индекс, проверить состояние ячейки и сравнить ключ. Библиотечная `unordered_flat_map` является более универсальной и промышленно оптимизированной структурой, но в таком простом сценарии ее дополнительные механизмы, более сложная внутренняя организация и универсальность могут давать большой константный overhead.

Таким образом, benchmark-case `Get existing sequential` показывает стоимость успешного поиска в наиболее благоприятном сценарии. Все реализации сохраняют ожидаемый линейный рост для серии из `N` поисков. Основные различия между ними связаны не с асимптотикой, а с постоянными факторами: стоимостью вызовов и проверок, уровнем реализации, внутренним layout памяти и степенью универсальности контейнера. Для собственной реализации этот сценарий особенно выгоден, поскольку последовательные ключи практически не создают коллизий, а операция поиска не вызывает перестроения таблицы.

Get existing random order

В данном benchmark-case измеряется время поиска уже существующих элементов, но в случайном порядке. Для каждого значения **N** таблица заранее заполняется набором случайных уникальных ключей. Затем этот же набор ключей перемешивается, и в измеряемой части benchmark-а выполняются операции поиска по всем существующим ключам. Таким образом, измеряется только стоимость успешного поиска, но порядок обращений к элементам становится случайным.



note

1 4. Get existing random order

2

	N	cpp_HashTable	unordered_flat_map	python_HashTable	dict
5	1000000	~0.413 s	~0.192 s	~2.440 s	~0.936 s
6	2000000	~0.889 s	~0.454 s	~4.979 s	~2.102 s
7	5000000	~2.495 s	~1.271 s	~14.438 s	~5.927 s
8	10000000	~4.399 s	~2.022 s	~30.442 s	~12.864 s
9	15000000	~6.547 s	~3.372 s	~44.351 s	~19.527 s

По графику видно, что все реализации сохраняют близкий к линейному рост при увеличении **N**. Это соответствует ожидаемой оценке: один успешный поиск в хеш-таблице в среднем выполняется за $O(1)$, поэтому серия из **N** поисков имеет сложность $O(N)$. Однако по сравнению с предыдущим кейсом **Get existing sequential** времена выполнения заметно возрастают почти у всех реализаций.

Главное отличие данного случая состоит в том, что нарушается локальность обращений. В предыдущем

benchmark-case ключи искались последовательно: `0, 1, 2, ..., N - 1`. Если элементы расположены в таблице достаточно регулярно, процессор может эффективнее использовать кэш: при обращении к одной ячейке в cache line попадают и соседние данные. В данном случае запросы выполняются в случайном порядке, поэтому обращения распределены по разным участкам таблицы. Это увеличивает количество cache miss и делает доступ к памяти дороже.

Кроме того, сами ключи были вставлены случайно. В отличие от последовательной вставки, где при `hash(key) = key` ключи почти идеально попадают в разные позиции, случайные ключи создают реальные коллизии. Поэтому при поиске существующего элемента иногда требуется пройти несколько ячеек probing-последовательности, прежде чем нужный ключ будет найден. Асимптотика при этом остается средней $O(1)$, но постоянная стоимость одного поиска увеличивается.

Особенно заметно это у собственной Python-реализации. Каждый поиск выполняется через метод `get`, который вызывает поиск слота, проверяет состояния ячеек, сравнивает ключи и обращается к внутренним массивам. В этом кейсе к обычным накладным расходам Python добавляются менее предсказуемые обращения к памяти и возможные дополнительные шаги probing. Поэтому python HashTable остается самой медленной реализацией.

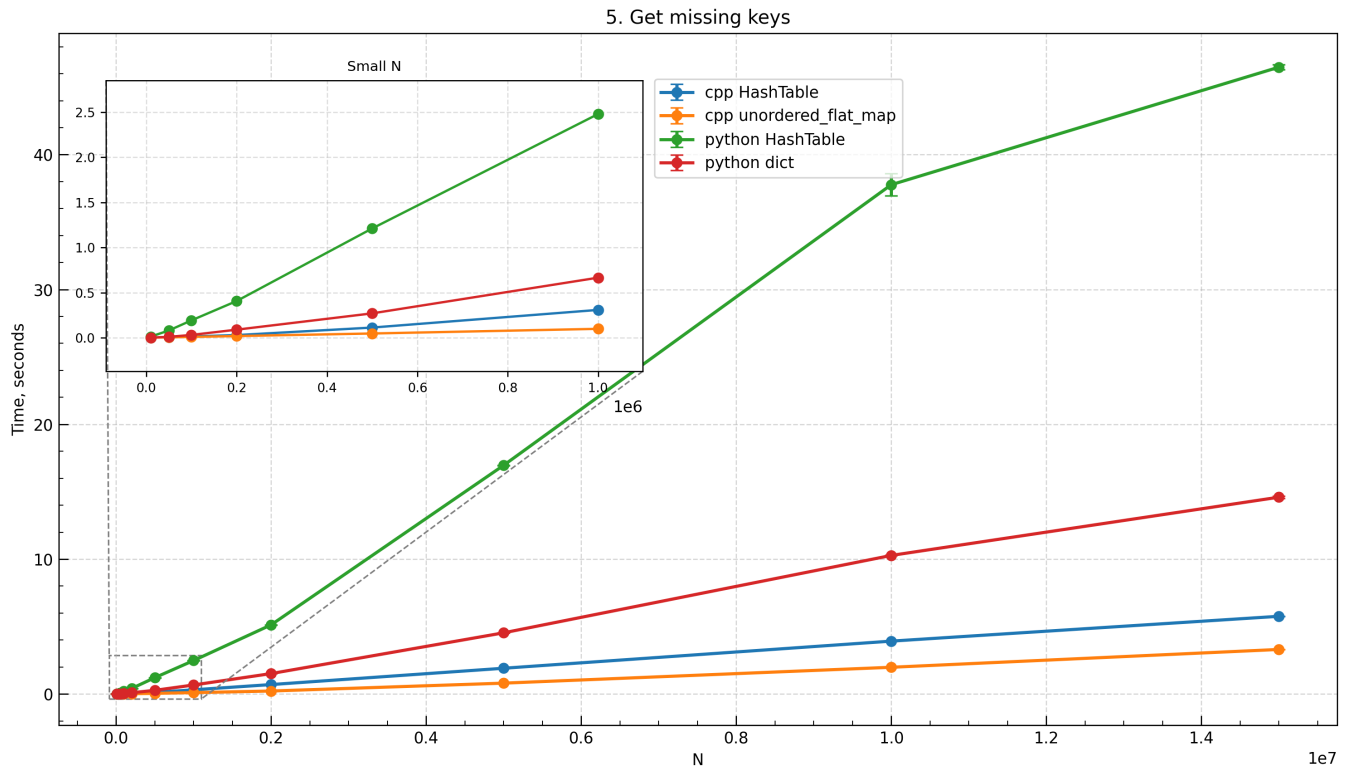
Интересно, что встроенный `dict` в этом кейсе становится заметно медленнее, чем в `Get existing sequential`. Это связано не с изменением асимптотики, а с характером доступа к данным. При случайном порядке поиска хуже работает кэш процессора: обращения идут не к соседним областям памяти, а к различным частям большой таблицы. Несмотря на это, `dict` всё равно существенно быстрее собственной Python-реализации, поскольку основная логика поиска выполняется внутри оптимизированной реализации CPython.

Среди C++-реализаций `unordered_flat_map` показывает лучший результат. В отличие от предыдущего кейса, где собственная C++ HashTable была быстрее при последовательном поиске, здесь библиотечная реализация выигрывает. Это объясняется тем, что случайный порядок поиска и случайное распределение ключей сильнее проверяют качество внутренней организации таблицы: layout памяти, стратегию probing, обработку коллизий и эффективность работы с кэшем. В таких условиях промышленная flat hash map получает преимущество над более простой учебной реализацией.

Таким образом, benchmark-case `Get existing random order` показывает, что даже успешный поиск существующих ключей может заметно замедляться при случайном порядке обращений. Общая сложность остается линейной для серии из `N` запросов, но константные факторы возрастают из-за ухудшения кэш-локальности и появления реальных probing-последовательностей. Этот кейс хорошо демонстрирует, что для хеш-таблиц важна не только асимптотика операции поиска, но и характер распределения ключей и порядок доступа к данным.

Get missing keys

В данном benchmark-case измеряется время поиска отсутствующих ключей. Для каждого значения N хеш-таблица заранее заполняется набором существующих случайных ключей. После этого генерируется другой набор из N ключей, которые гарантированно отсутствуют в таблице, и в измеряемой части benchmark-a выполняется поиск этих отсутствующих ключей.



note

5. Get missing keys

	N	cpp_HashTable	unordered_flat_map	python_HashTable	dict
1	1000000	~0.309 s	~0.097 s	~2.468 s	~0.660 s
2	2000000	~0.697 s	~0.217 s	~5.126 s	~1.514 s
3	5000000	~1.906 s	~0.803 s	~16.966 s	~4.538 s
4	10000000	~3.921 s	~2.016 s	~36.934 s	~10.249 s
5	15000000	~5.759 s	~3.303 s	~46.321 s	~14.607 s

По графику видно, что все реализации сохраняют близкий к линейному рост при увеличении N . Это ожидаемо: один поиск в хеш-таблице в среднем выполняется за $O(1)$, поэтому серия из N запросов имеет ожидаемую сложность $O(N)$. Однако поиск отсутствующих ключей обычно оказывается дороже, чем поиск существующих.

Главная причина состоит в том, что при успешном поиске структура может остановиться сразу после нахождения нужного ключа. При неуспешном поиске необходимо убедиться, что ключа в таблице действительно нет.

Для реализации с open addressing это означает, что probing продолжается до тех пор, пока не будет найдена пустая ячейка `EMPTY`, которая доказывает, что ключ отсутствует. Поэтому поиск отсутствующего элемента часто требует больше проверок, чем поиск существующего.

В данном кейсе таблица заполнена случайными ключами, а отсутствующие ключи также распределены случайно относительно текущего состояния таблицы. Поэтому обращения к ячейкам менее локальны, чем в последовательных сценариях, а часть запросов проходит через несколько занятых позиций перед тем, как встретить пустую ячейку. Это увеличивает среднюю длину probing-последовательности и, соответственно, константную стоимость одного поиска.

Особенно заметно это у собственной Python-реализации. Поиск отсутствующего ключа вызывает метод `get`, внутри которого выполняется поиск слота, проверки состояний ячеек, сравнения ключей и переходы по probing-последовательности. Так как ключ не находится, поиск чаще проходит дальше, чем при успешном попадании. В результате к обычным накладным расходам Python добавляется дополнительная стоимость неуспешного probing, поэтому линия python HashTable остается самой высокой.

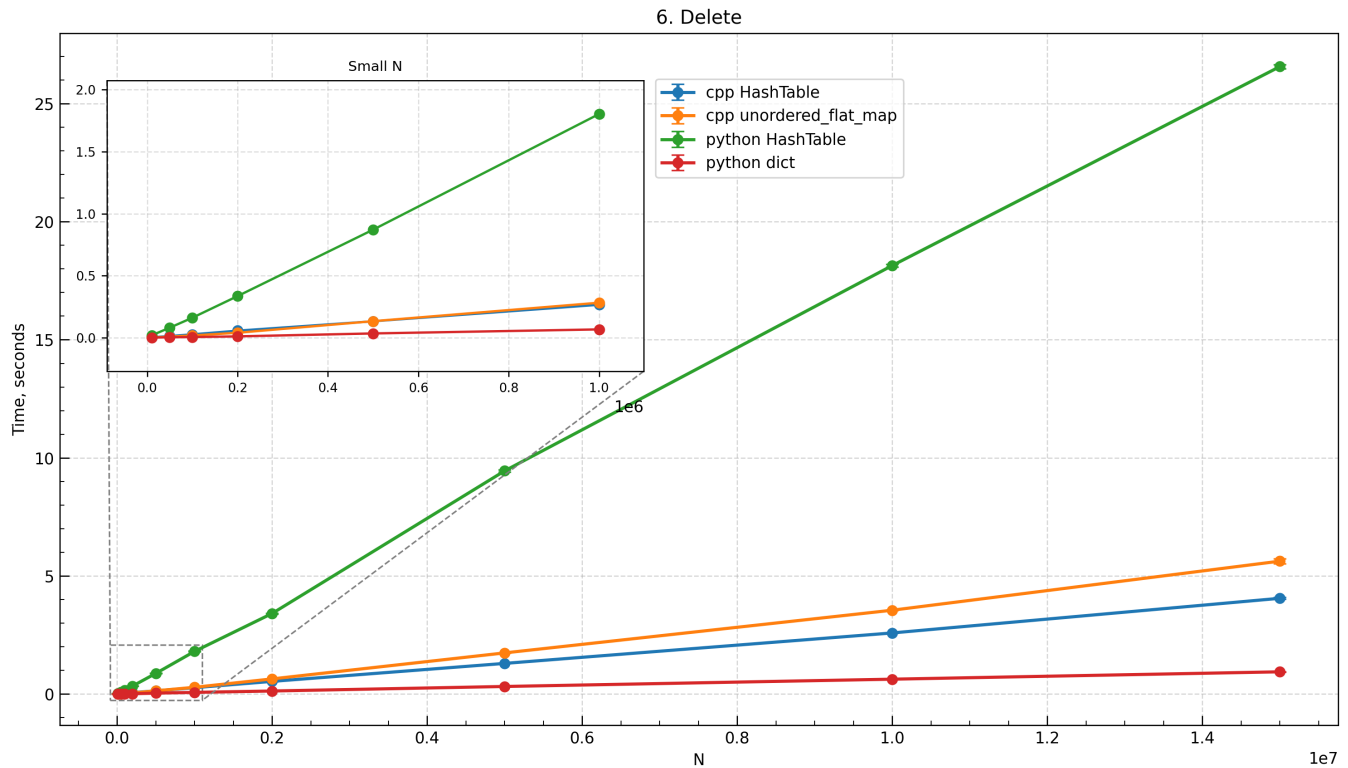
Встроенный `dict` также работает медленнее, чем в простом последовательном поиске существующих ключей. Это нормально: отрицательный поиск требует доказать отсутствие ключа, а не просто найти его. Тем не менее `dict` значительно быстрее собственной Python-реализации, поскольку основная логика поиска выполняется внутри оптимизированной реализации CPython.

Среди C++-реализаций `unordered_flat_map` показывает лучший результат. Это объясняется тем, что поиск отсутствующих ключей хорошо проверяет качество внутренней стратегии probing и работу с памятью. Библиотечная flat hash map эффективнее обрабатывает такие отрицательные запросы: у нее меньше постоянные расходы на одну проверку, лучше организован layout памяти и оптимизирована работа с группами ячеек. Собственная C++-реализация остается значительно быстрее Python-реализации, но проигрывает библиотечной из-за более простой внутренней организации.

Таким образом, benchmark-case `Get missing keys` показывает более тяжелый вариант операции поиска. Асимптотически серия из N запросов по-прежнему имеет ожидаемую сложность $O(N)$, однако отрицательный поиск увеличивает константную стоимость операции, потому что структуре необходимо пройти probing-последовательность до пустой ячейки и убедиться в отсутствии ключа. Поэтому данный кейс сильнее нагружает механизм разрешения коллизий и работу с памятью, чем поиск существующих элементов.

Delete

В данном benchmark-case измеряется время удаления элементов из хеш-таблицы. Для каждого значения N структура данных заранее заполняется ключами $0, 1, 2, \dots, N - 1$, после чего в измеряемой части benchmark-a выполняется последовательное удаление всех этих ключей. Таким образом, измеряется не построение таблицы, а только стоимость операций удаления из уже заполненной структуры.



note

1 6. Delete

	N	cpp_HashTable	unordered_flat_map	python_HashTable	dict
5	1000000	~0.264 s	~0.282 s	~1.810 s	~0.067 s
6	2000000	~0.527 s	~0.638 s	~3.397 s	~0.125 s
7	5000000	~1.292 s	~1.740 s	~9.419 s	~0.318 s
8	10000000	~2.573 s	~3.545 s	~18.145 s	~0.630 s
9	15000000	~4.015 s	~5.619 s	~26.548 s	~0.939 s

По графику видно, что все реализации в целом сохраняют близкий к линейному рост при увеличении N . Это ожидаемо: удаление одного элемента из хеш-таблицы в среднем выполняется за $O(1)$, поэтому серия из N удалений имеет ожидаемую сложность $O(N)$. Однако по сравнению с простым успешным поиском операция удаления обычно имеет большую константную стоимость, поскольку она не только находит элемент, но и изменяет внутреннее состояние таблицы.

Особенность удаления в хеш-таблице с open addressing состоит в том, что элемент нельзя просто заменить

на пустую ячейку. Если сразу поставить состояние `EMPTY`, это может нарушить корректность последующего поиска: probing-последовательность для других ключей может преждевременно остановиться. Поэтому в собственной реализации используется специальное состояние `DELETED`, то есть tombstone. Такая ячейка считается удаленной, но сохраняет цепочку поиска.

Из-за этого удаление включает несколько этапов: сначала нужно найти ключ в таблице, затем пометить соответствующую ячейку как `DELETED`, уменьшить размер структуры и обновить счетчик удаленных ячеек. Если tombstone становится слишком много, производительность будущих операций ухудшается, поэтому реализация выполняет rebuild: создает новую таблицу того же размера и заново вставляет в нее оставшиеся элементы, уже без удаленных ячеек.

Именно эта логика делает удаление более тяжелым benchmark-case для собственной реализации. При большом числе последовательных удалений tombstones накапливаются быстро, поэтому время включает не только стоимость самих операций `delete`, но и стоимость периодических перестроений таблицы. Такие перестроения являются редкими дорогими операциями: они не меняют амортизированную сложность всей серии, но увеличивают фактическое время выполнения.

Наиболее медленной снова оказывается собственная реализация на Python. Для каждого удаления она выполняет поиск слота, проверки состояний, сравнение ключей, изменение служебных счетчиков и установку состояния `DELETED` на уровне Python-кода. Кроме того, при срабатывании rebuild оставшиеся элементы заново переносятся через Python-логику вставки. Поэтому на больших `N` накопление накладных расходов становится особенно заметным.

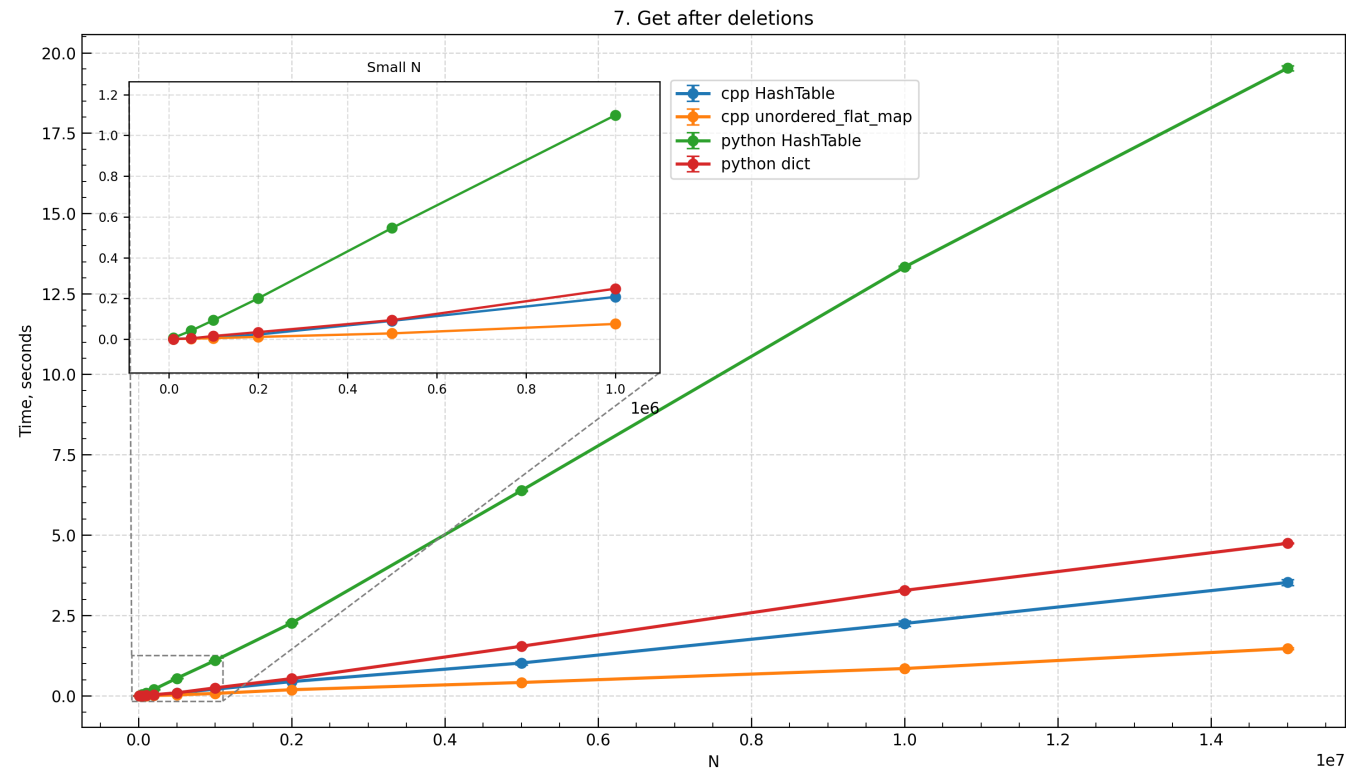
Встроенный `dict` показывает значительно меньшие времена. Он также должен корректно обрабатывать удаление и внутреннее состояние таблицы, однако все эти действия выполняются внутри оптимизированной реализации CPython. Поэтому стоимость одного удаления и стоимость обслуживания внутренней структуры оказываются намного ниже, чем у пользовательской Python-реализации.

Среди C++-реализаций `unordered_flat_map` оказывается медленнее собственной C++ HashTable на малых и больших размерах по графику заметно не всегда одинаково близко, но в целом обе реализации находятся существенно ниже Python HashTable. Разница между ними определяется уже не интерпретацией, а деталями обработки удаления: как хранятся метаданные, как помечаются удаленные элементы, когда выполняется очистка, сколько проверок требуется на одну операцию и насколько удачно используется кэш. Библиотечная реализация может иметь более сложную универсальную логику удаления, тогда как собственная реализация в данном последовательном сценарии работает достаточно прямолинейно.

Таким образом, benchmark-case `Delete` показывает, что удаление в хеш-таблице с open addressing сложнее, чем простое успешное обращение по ключу. Общий рост времени остается близким к линейному, но константная стоимость операции увеличивается из-за необходимости поддерживать tombstones и периодически очищать таблицу. Этот кейс хорошо демонстрирует влияние не только базовой асимптотики, но и служебной логики структуры данных: удаления, счетчиков, состояний ячеек и rebuild.

Get after deletions

В данном benchmark-case измеряется время поиска существующих ключей после предварительных удалений. Для каждого значения `N` таблица сначала заполняется набором случайных уникальных ключей. Затем из нее удаляется примерно половина элементов, а в измеряемой части benchmark-a выполняется поиск оставшихся ключей. Таким образом, данный тест проверяет не просто успешный поиск, а поиск в таблице, в которой до этого уже происходили удаления.



note

1 7. Get after deletions

2

3	N	cpp_HashTable	unordered_flat_map	python_HashTable	dict
4					
5	1000000	~0.208 s	~0.076 s	~1.103 s	~0.248 s
6	2000000	~0.443 s	~0.190 s	~2.277 s	~0.538 s
7	5000000	~1.067 s	~0.412 s	~6.378 s	~1.552 s
8	10000000	~2.173 s	~0.836 s	~13.361 s	~3.282 s
9	15000000	~3.669 s	~1.464 s	~19.566 s	~4.739 s

По графику видно, что все реализации сохраняют близкий к линейному рост при увеличении `N`. Это соответствует ожидаемой оценке: один поиск существующего ключа в хеш-таблице в среднем выполняется за $O(1)$, поэтому серия из большого числа поисков имеет линейную сложность относительно количества запросов.

Главная особенность данного benchmark-case связана с тем, что перед поиском из таблицы удаляется часть элементов. В реализации с open addressing удаление не может просто заменить ячейку на `EMPTY`, так как

это может нарушить корректность последующих поисков. Поэтому удаленные элементы оставляют после себя специальные ячейки `DELETED`, или tombstones. Они сохраняют probing-последовательности, но ухудшают производительность: поиск может проходить через удаленные ячейки так же, как через занятые, пока не найдет нужный ключ или не встретит пустую ячейку.

В собственной реализации предусмотрена очистка tombstones через rebuild. Поэтому после большого числа удалений часть удаленных ячеек может быть устранена перестроением таблицы. Тем не менее сам сценарий остается тяжелее обычного успешного поиска: структура уже пережила массовое удаление, а расположение оставшихся элементов и служебных состояний становится менее “чистым”, чем в таблице, куда элементы только вставлялись.

Наиболее медленной остается собственная реализация на Python. Даже если часть tombstones очищается rebuild-ом, каждый поиск выполняется через Python-код: вызывается метод `get`, выполняется `_find_slot`, проверяются состояния ячеек, сравниваются ключи и выполняются переходы по probing-последовательности. При большом количестве запросов эти накладные расходы дают заметный разрыв по времени.

Интересно, что времена в этом кейсе меньше, чем в некоторых других сценариях поиска со случайным доступом, потому что после удаления половины элементов измеряется поиск только по оставшимся ключам. То есть фактическое количество поисковых запросов меньше, чем исходное `N`: ищутся не все исходные элементы, а только те, которые не были удалены. Поэтому при сравнении с другими benchmark-case важно учитывать, что размер `N` здесь задает исходный размер таблицы, а не обязательно точное число операций поиска в измеряемой части.

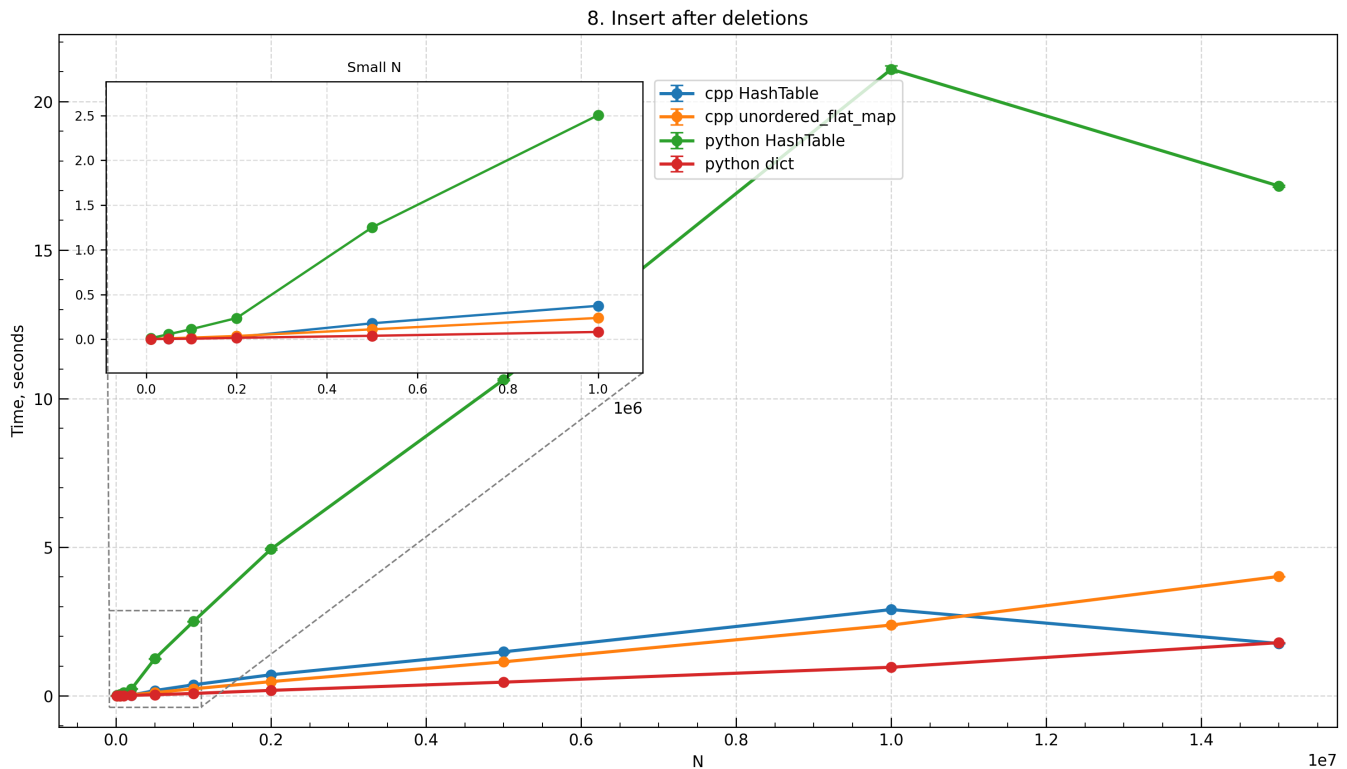
Среди C++-реализаций `unordered_flat_map` показывает лучший результат. Это ожидаемо для сценария после удалений: библиотечная реализация лучше оптимизирована для обслуживания внутреннего состояния таблицы, обработки удаленных ячеек и сохранения хорошей производительности после изменений структуры. Собственная C++-реализация также работает значительно быстрее Python-реализации, но уступает библиотечной из-за более простой стратегии хранения и обработки служебных состояний.

Встроенный `dict` также демонстрирует хорошую производительность, хотя после удалений его время выше, чем в простом последовательном поиске. Это связано с тем, что удаление меняет внутреннее состояние таблицы и может увеличивать служебную работу при последующих поисках. Однако благодаря низкоуровневой реализации CPython `dict` остается существенно быстрее собственной Python-реализации.

Таким образом, benchmark-case `Get after deletions` показывает влияние удалений на последующие операции поиска. Асимптотически серия поисков остается линейной, но наличие удаленных ячеек, перестроения таблицы и менее регулярное расположение данных увеличивают константные расходы. Этот кейс важен, потому что он проверяет хеш-таблицу не в идеальном состоянии после одних только вставок, а после изменения структуры удалениями.

Insert after deletions

В данном benchmark-case измеряется время вставки новых элементов после предварительных удалений. Для каждого значения N таблица сначала заполняется последовательными ключами $0, 1, 2, \dots, N - 1$. Затем удаляется примерно половина элементов, после чего в измеряемой части benchmark-а вставляются новые ключи. Таким образом, этот тест проверяет не вставку в полностью пустую таблицу, а вставку в структуру, которая уже содержит часть данных и имеет следы предыдущих удалений.



note

1 8. Insert after deletions

	N	cpp_HashTable	unordered_flat_map	python_HashTable	dict
5	1000000	~0.375 s	~0.231 s	~2.502 s	~0.081 s
6	2000000	~0.709 s	~0.474 s	~4.915 s	~0.191 s
7	5000000	~1.475 s	~1.143 s	~10.643 s	~0.469 s
8	10000000	~2.891 s	~2.380 s	~20.961 s	~0.959 s
9	15000000	~1.757 s	~4.030 s	~17.170 s	~1.788 s

По графику видно, что большинство реализаций в целом сохраняют близкое к линейному поведение, однако этот кейс выглядит менее ровным, чем обычная вставка. Особенно заметно это у собственной реализации: время при $N = 10\,000\,000$ оказывается больше, чем при $N = 15\,000\,000$. Это не означает нарушения асимптотической оценки. Такой эффект связан с конкретным состоянием внутренней таблицы после последовательности вставок и удалений.

При `N = 10 000 000` размер таблицы оказывается подобран так, что итоговая загрузка находится почти у порога расширения: `load factor ≈ 0.596`, тогда как расширение таблицы происходит при значении около `0.6`. Поэтому значительная часть операций вставки выполняется в таблице, которая уже почти достигла критической заполненности. В таком состоянии увеличивается вероятность более длинного probing, а также сильнее проявляется влияние удаленных ячеек.

При `N = 15 000 000` ситуация оказывается иной: из-за предыдущего расширения вместимость таблицы уже больше, и фактический `load factor` составляет примерно `0.447`. Несмотря на больший исходный размер данных, таблица имеет больше свободного пространства относительно своей `capacity`. Поэтому новые вставки происходят в более “разреженной” таблице, что снижает среднюю длину probing и уменьшает стоимость операций.

Главная особенность данного benchmark-case — наличие удаленных ячеек. В хеш-таблице с open addressing удаленный элемент обычно оставляет после себя `DELETED`-ячейку, или tombstone. Такая ячейка нужна для корректности поиска, но она ухудшает последующие операции: при вставке таблица должна учитывать, что удаленный слот можно переиспользовать, но при этом probing не может останавливаться на нем так же, как на полностью пустой ячейке.

Из-за этого вставка после удалений может быть дороже обычной вставки. Структура должна не только найти место для нового ключа, но и пройти probing-последовательность с учетом `OCCUPIED`, `DELETED` и `EMPTY` ячеек. Если tombstones накопилось много, средняя длина probing может увеличиваться. Поэтому такие сценарии сильнее проверяют качество обработки удалений и политику очистки таблицы.

У собственной Python-реализации эффект выражен особенно сильно, потому что вся логика поиска слота, учета удаленных ячеек, проверки порогов и возможного rebuild выполняется на уровне Python-кода. Поэтому любая дополнительная служебная работа заметно увеличивает время выполнения.

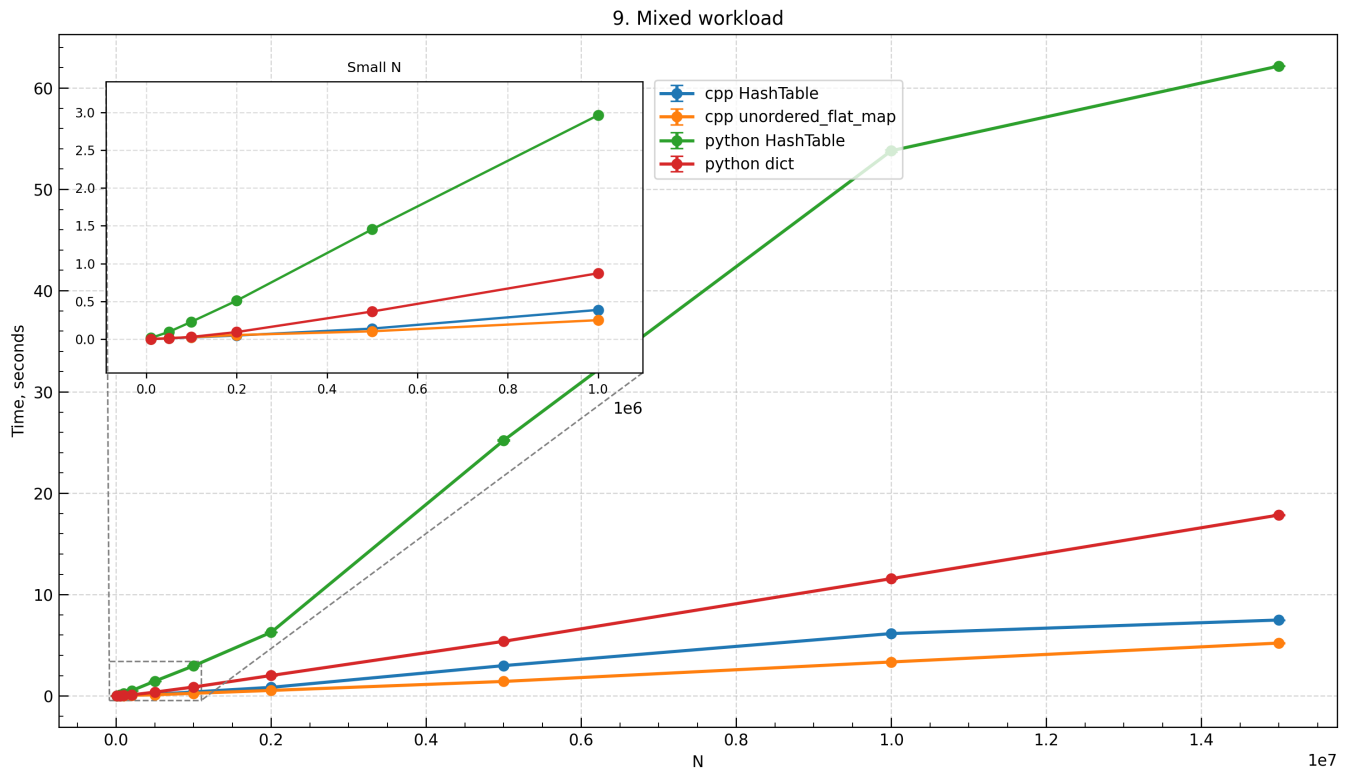
Встроенный `dict` показывает более стабильное поведение. Он также должен обслуживать удаленные элементы и внутреннее состояние таблицы, но делает это внутри оптимизированной реализации CPython. Поэтому рост времени остается более плавным, а абсолютные значения значительно ниже, чем у собственной Python-реализации.

Среди C++-реализаций заметно, что `unordered_flat_map` на больших размерах оказывается быстрее собственной C++ HashTable. Этот сценарий сильнее нагружает обработку удаленных ячеек, повторное использование слотов и политику роста/очистки таблицы. Библиотечная реализация лучше оптимизирована для таких ситуаций, тогда как собственная реализация использует более простую логику обработки tombstones и rebuild.

Таким образом, benchmark-case `Insert after deletions` показывает, что вставка в таблицу после удалений является более сложным сценарием, чем вставка в пустую структуру. Асимптотически серия вставок остается линейной, но фактическое время зависит от количества tombstones, моментов срабатывания rebuild, текущей заполненности таблицы и качества повторного использования удаленных слотов. Поэтому в этом кейсе график может быть менее гладким, а различия между реализациями становятся более связанными с внутренней политикой обслуживания таблицы, чем с базовой операцией вставки.

Mixed workload

В данном benchmark-case измеряется работа хеш-таблицы в смешанном сценарии, где последовательно выполняются разные типы операций: поиск, вставка и удаление. В отличие от предыдущих тестов, здесь структура данных не проверяется только на одном действии. Вместо этого benchmark имитирует более реалистичную нагрузку, при которой таблица постоянно изменяется: часть ключей добавляется, часть удаляется, а часть используется для поиска.



note

9. Mixed workload

	N	cpp_HashTable	unordered_flat_map	python_HashTable	dict
5	1000000	~0.389 s	~0.252 s	~2.966 s	~0.865 s
6	2000000	~0.837 s	~0.532 s	~6.266 s	~1.994 s
7	5000000	~2.959 s	~1.417 s	~25.204 s	~5.384 s
8	10000000	~6.137 s	~3.379 s	~53.785 s	~11.583 s
9	15000000	~7.469 s	~5.245 s	~62.155 s	~17.880 s

По графику видно, что все реализации в целом сохраняют рост, близкий к линейному. Это ожидаемо, поскольку каждая отдельная операция хеш-таблицы — вставка, поиск и удаление — в среднем имеет амортизированную сложность $O(1)$. Следовательно, серия из N смешанных операций имеет ожидаемую сложность $O(N)$.

Однако данный сценарий является более сложным, чем отдельные benchmark-case на вставку или поиск. Здесь одновременно проявляются несколько факторов: случайный порядок обращений, наличие успешных и

неуспешных поисков, добавление новых ключей, удаление существующих ключей, накопление `DELETED` - ячеек и возможные перестроения таблицы. Поэтому `mixed workload` лучше показывает поведение структуры данных в условиях, близких к реальному использованию.

Наиболее медленной снова оказывается собственная реализация на Python. Это ожидаемо, потому что каждая операция выполняется через Python-код: вызываются методы `put`, `get`, `delete`, выполняется поиск слота, проверки состояний ячеек, сравнения ключей, обработка исключений при удалении отсутствующего ключа и возможные перестроения таблицы. В смешанном сценарии эти накладные расходы суммируются, поэтому разрыв с остальными реализациями остается большим.

Встроенный `dict` показывает существенно меньшие времена выполнения. Несмотря на то что он также обрабатывает вставки, поиски, удаления и внутренние изменения таблицы, основная работа выполняется внутри оптимизированной реализации CPython. Поэтому стоимость одной операции значительно ниже, чем у пользовательской Python-реализации.

Среди C++-реализаций `unordered_flat_map` показывает лучший результат. Это особенно заметно именно в смешанном сценарии, поскольку он нагружает не один простой путь выполнения, а всю внутреннюю механику контейнера: вставку, поиск, удаление, повторное использование ячеек, работу с заполненностью и очистку служебного состояния. Библиотечная реализация лучше оптимизирована для таких неоднородных нагрузок, поэтому имеет меньшие постоянные расходы.

Собственная C++ HashTable также сохраняет хорошее поведение и остается значительно быстрее собственной Python-реализации. Однако в `mixed workload` она уступает `unordered_flat_map`, поскольку ее внутренняя логика проще и менее оптимизирована для сложных последовательностей операций. В таких условиях важны не только базовая асимптотика, но и детали обработки удалений, `probing`, `rebuild`, `layout` памяти и кэш-локальность.

Таким образом, benchmark-case `Mixed workload` является наиболее комплексным тестом для хеш-таблицы. Он подтверждает ожидаемое линейное поведение серии операций, но одновременно показывает, насколько сильно практическая производительность зависит от качества реализации внутренних механизмов. В отличие от простых сценариев, здесь одновременно проявляются эффекты вставки, поиска, удаления, `tombstones`, `resize/rebuild` и случайного доступа к данным. Поэтому данный benchmark хорошо подходит для итогового сравнения реализаций в условиях, близких к реальной нагрузке.

Общий вывод по benchmark-ам Hash Table

По результатам benchmark-ов видно, что все реализации в большинстве случаев демонстрируют ожидаемое поведение с точки зрения асимптотики. Серии из N операций вставки, поиска и удаления в среднем дают рост, близкий к линейному. Это соответствует теоретической модели хеш-таблицы: базовые операции имеют среднюю или амортизированную сложность $O(1)$, поэтому последовательность из N операций имеет ожидаемую сложность $O(N)$.

При этом эксперименты показывают, что одинаковая асимптотика не означает одинаковую практическую производительность. Разница между реализациями определяется константными факторами: уровнем реализации, накладными расходами языка, организацией памяти, кэш-локальностью, обработкой коллизий, политикой расширения таблицы и эффективностью работы с удаленными ячейками.

На простых синтетических сценариях собственная C++-реализация иногда оказывается быстрее `unordered_flat_map`. Особенно это заметно в кейсах, где используются последовательные целочисленные ключи и операции выполняются в регулярном порядке. В таких условиях собственная реализация работает в почти идеальном режиме: ключи хорошо ложатся в таблицу, probing почти не используется, доступ к данным достаточно предсказуем, а сама операция сводится к небольшому числу простых действий. Более универсальная библиотечная структура в таких случаях может иметь больший постоянный overhead, связанный со своей внутренней организацией и поддержкой более широкого набора сценариев.

Однако в более реалистичных benchmark-ах ситуация меняется. В сценариях со случайными ключами, случайным порядком поиска, отсутствующими ключами, удалениями и смешанной нагрузкой `boost::unordered_flat_map` показывает себя значительно сильнее. Эти кейсы лучше проверяют не только базовую операцию, но и качество всей внутренней реализации: стратегию probing, layout памяти, обработку коллизий, повторное использование удаленных ячеек, политику роста и устойчивость к неоднородной нагрузке.

Именно поэтому `boost::unordered_flat_map` можно рассматривать как более надежную и промышленно оптимизированную реализацию. Она не обязательно выигрывает в специально благоприятных синтетических случаях, но лучше сохраняет производительность в сложных и приближенных к реальному использованию сценариях. Это важное наблюдение: простая учебная реализация может быть конкурентоспособной на “идеальных” данных, однако библиотечная реализация выигрывает за счет устойчивости и оптимизации большого числа деталей.

Собственная Python-реализация во всех benchmark-ах существенно уступает `dict`. Это ожидаемый результат: даже при той же алгоритмической идее операции собственной таблицы выполняются на уровне Python-кода, тогда как `dict` использует низкоуровневую реализацию внутри CPython. Поэтому разрыв между ними связан не только с устройством хеш-таблицы, но и с накладными расходами интерпретируемого выполнения, объектной моделью Python и стоимостью обращений к пользовательским структурам.

Встроенный `dict` стабильно показывает высокую производительность и во многих кейсах оказывается одной из самых быстрых реализаций. Это подтверждает, что стандартные контейнеры Python нельзя воспринимать как обычный Python-код: их производительность определяется низкоуровневой реализацией, оптимизированной под частые сценарии использования.